

Data Breach Analysis — Solution & Marking Guide

English solution for `DataBreach_Lab.md`. Exact numbers depend on the dump provided; this gives the method, the expected results per step, and what to look for when marking.

Note: how to **obtain** the dump (dark-web sourcing) is a separate, sensitive topic kept out of this public solution. Provide the dump directly to students for this lab.

Posture

Defensive impact analysis. Import locally, **read-only**, map an unknown schema, quantify the exposure, reason about GDPR. Forbidden: re-identifying or contacting victims, cracking passwords. The reference dataset is a social-network CMS dump (UNA/BoonEx family, several hundred tables), typical of a community/dating site — so expect user accounts, public profiles and private messaging tables.

Expected results per step

Step 0 — Import & sanity check - `grep -c '^CREATE TABLE' dump.sql` returns **hundreds** of tables → signature of a full social-network CMS, not a single-app DB. - The dump header (`-- MariaDB dump ... Database: <name>`) reveals the platform and often a shared-hosting prefix (cPanel style). That tells you which tables to expect.

Step 1 — Connect & discover the schema - List tables via `information_schema.tables` with `table_rows`. Identify core tables by name (`%user%`, `%profile%`, `%message%`) and by row count. - Watch for dates stored as **Unix timestamps** (integers), not datetimes. - Deliverable: a documented map of 3–4 key tables (columns, types, a sample).

Step 2 — User accounts - Total accounts; active (a status column); verified emails (a verification column). - Monthly registrations via `FROM_UNIXTIME(ts)` or `pd.to_datetime(ts, unit='s')`. - Top 10 email domains — usually dominated by gmail/hotmail/yahoo; note any corporate or regional domains.

Step 3 — Profiles & joins - Join accounts ↔ public profiles on the foreign key; **validate on samples** (don't confuse account id with profile id). - Demographics by gender. - Ages from birth dates, bucketed `<18 / 18–25 / 26–40 / 41–60 / 60+`, handling invalid dates (`0000-00-00`, future dates, nulls). - Top 10 most-viewed profiles.

Step 4 — Messaging - Total conversations and messages; monthly message volume; top 10 conversations (by volume, with titles); top 10 most active users.

Step 5 — Dashboard, export, synthesis - A consolidated KPI DataFrame (accounts, active, verified, profiles, conversations, messages, earliest/latest registration). - Export one aggregated dataset (e.g. monthly registrations) to a clean CSV. - A 5–10 line synthesis: peak periods & trends, typical demographic, and the **GDPR/privacy vulnerabilities** found.

GDPR & security points to expect

- Direct personal data (email, name, date of birth, gender) → GDPR art. 4; on a dating/community site, some data may be **special category** (art. 9).
- Password storage (bonus, read-only): identify the algorithm by hash signature/length (MD5 = 32 hex; bcrypt = \$2y\$...; SHA-1 = 40 hex) and whether a salt is present. Comment on robustness **without cracking anything**.
- In a real incident: notify the DPA within 72h and inform data subjects.

Reference query patterns

```
-- table count
SELECT COUNT(*) FROM information_schema.tables WHERE table_schema = DATABASE();
-- tables by estimated rows
SELECT table_name, table_rows FROM information_schema.tables
WHERE table_schema = DATABASE() ORDER BY table_rows DESC;
-- monthly registrations (Unix timestamp column)
SELECT DATE_FORMAT(FROM_UNIXTIME(created), '%Y-%m') ym, COUNT(*) n
FROM users GROUP BY ym ORDER BY ym;
-- top email domains
SELECT SUBSTRING_INDEX(email, '@', -1) dom, COUNT(*) n
FROM users GROUP BY dom ORDER BY n DESC LIMIT 10;
```

```
import os, pandas as pd
from sqlalchemy import create_engine, inspect
eng = create_engine(os.environ["DATABASE_URL"]) # never hard-code creds
tables = inspect(eng).get_table_names()
reg = pd.read_sql("SELECT created FROM users", eng)
reg["month"] = pd.to_datetime(reg["created"], unit="s").dt.to_period("M")
reg.groupby("month").size().to_csv("monthly_registrations.csv")
```

Common pitfalls (deduct / coach)

Pitfall	Fix
Hard-coding DB credentials	read from DATABASE_URL env var
Reading Unix timestamps as seconds-since-nothing	FROM_UNIXTIME / pd.to_datetime(unit='s')
Joining on the wrong id (account vs profile)	validate on sample rows
Leaving personal data unmasked in the report	anonymise emails/names
Slow import of a large dump	use a dedicated MariaDB container (see the Docker & n8n Lab)

Marking (100 pts)

Item	Pts	What earns the points
Import & schema discovery	20	table count, core tables found, schema documented
User analysis	20	volumetrics, monthly trend, top domains
Profiles & joins	20	correct join, demographics, age brackets, missing-value handling
Messaging	15	volumetrics, top conversations/users
Dashboard & synthesis	20	KPI summary, CSV export, GDPR reasoning
Rigour	5	anonymised output, env-var credentials, read-only